

Matriz Comparativa y Optimización de Costos

1. Objetivo

Esta matriz documenta la estrategia de selección y optimización de tecnologías utilizada en la Mesa de Ayuda Inteligente. El objetivo es asignar cada tarea al componente adecuado, reducir procesamiento innecesario y mantener controlado el consumo de recursos.

2. Solución implementada

El flujo utiliza n8n como orquestador, Airtable como base de datos, un AI Agent con Groq Chat Model para procesamiento de lenguaje natural, Simple Vector Store para recuperación de conocimiento mediante RAG y Gmail como canal de salida y validación Human-in-the-Loop (HITL).

3. Matriz de decisión de costos

Tarea	Solución	Motivo	Optimización
Clasificación de tickets	Groq Chat Model + AI Agent	Interpretar el ticket y determinar área y prioridad.	Prompt estructurado + Structured Output.
Generación de resumen	Groq Chat Model	Convertir la descripción en un resumen operativo.	Respuesta breve y específica.
Consulta de conocimiento	Simple Vector Store + RAG	Recuperar información relevante.	Enviar solo contexto recuperado, no toda la base.
Decisión de revisión humana	AI Agent + Structured Output	Determinar si requiere intervención humana.	Campo booleano true/false.
Validación humana	Gmail + HITL	Esperar aprobación antes de una acción crítica.	Activar solo cuando sea necesario.
Registro y trazabilidad	Airtable	Guardar tickets, estados y ejecuciones.	Persistir solo información necesaria.
Procesamiento masivo — alternativa	API Batches	Agrupar tareas sin respuesta inmediata.	Optimizar grandes volúmenes.
Tareas simples — alternativa	Modelo liviano, ej. GPT-4o-mini	Resolver tareas estructuradas sencillas.	Elegir modelo según complejidad.
Lectura densa — alternativa	Claude	Reservarlo para documentos extensos.	No usar un modelo más costoso si no hace falta.

4. Optimización mediante Structured Output

El AI Agent devuelve únicamente los campos necesarios para continuar el flujo: área, prioridad, resumen y requiere_revision_humana. Esto reduce texto innecesario y facilita el mapeo hacia Airtable y las rutas de decisión.

Ejemplo de salida

```
{  
  "area": "Soporte Técnico",  
  "prioridad": "Alta",  
  "resumen": "El usuario reporta que no puede acceder a su cuenta debido a un problema con sus credenciales. Solicita recuperar el acceso con carácter urgente para poder continuar trabajando.",  
  "requiere_revision_humana": true  
}
```

5. Optimización mediante RAG

La arquitectura incorpora una base de conocimiento independiente. Los documentos se cargan mediante Edit Fields hacia Simple Vector Store. Otro Simple Vector Store se conecta al AI Agent como Tool para recuperar documentos relevantes.

Sin RAG, el sistema tendría que incorporar una cantidad mayor de información de la base en cada consulta. Con RAG, el agente recupera únicamente el contexto relevante para el ticket, reduciendo la información procesada en cada interacción.

Simple Vector Store – Carga de la base de conocimiento

El nodo Simple Vector Store ubicado después de Edit Fields se utiliza para cargar e indexar la información de la base de conocimiento que utilizará el agente de IA durante el procesamiento de los tickets.

El nodo Edit Fields prepara y estructura los datos que serán incorporados a la base vectorial, mientras que Simple Vector Store transforma esa información en una fuente consultable por el agente. De esta manera, el sistema puede almacenar conocimientos relevantes —como

procedimientos, soluciones frecuentes y documentación de soporte— y posteriormente recuperarlos mediante búsquedas semánticas.

Esta implementación permite separar dos funciones dentro del RAG:

- Simple Vector Store + Edit Fields: carga y prepara los documentos/conocimientos que conforman la base.
- Simple Vector Store conectado como herramienta del AI Agent: permite que el agente consulte esa base durante el análisis de cada ticket y utilice el contexto recuperado para generar una respuesta más precisa.

En resumen: este Simple Vector Store funciona como el repositorio de conocimiento del sistema, mientras que el segundo permite que el AI Agent consulte ese conocimiento cuando lo necesita.

6. Optimización del procesamiento

El filtro ¿Ticket Pendiente? evita procesar nuevamente tickets que no se encuentran en el estado esperado. Esto reduce ejecuciones innecesarias y ayuda a prevenir ciclos de reprocesamiento.

La revisión humana también funciona como mecanismo de control: no todos los tickets pasan por HITL, sino únicamente aquellos para los que el AI Agent establece `requiere_revision_humana = true`.

7. Selección de modelo según la tarea

La arquitectura sigue un principio de selección por complejidad: tareas simples pueden resolverse con modelos livianos; tareas de lectura extensa pueden justificar modelos con mayor capacidad contextual; y procesos masivos sin respuesta inmediata pueden beneficiarse de APIs de lotes.

En el workflow implementado se utiliza Groq Chat Model. Las referencias a GPT-4o-mini, Claude y Batches son alternativas contempladas por la consigna, no componentes actualmente utilizados en el workflow.

8. Ahorro estimado y limitaciones

La optimización esperada se produce principalmente por reducción de contexto, respuestas acotadas, procesamiento condicional y recuperación selectiva mediante RAG. No se asigna un porcentaje monetario de ahorro porque no se dispone de volumen histórico, consumo de tokens y precios

del proveedor suficientes para calcular una cifra confiable.

Para una medición posterior, se recomienda registrar por ejecución: tickets procesados, tokens de entrada y salida, modelo utilizado, consultas RAG, errores y porcentaje de tickets derivados a HITL.

9. Conclusión

La estrategia de costos de TechSupport Solutions combina selección de modelo según complejidad, Structured Output, RAG, procesamiento condicional y validación humana selectiva. Estas decisiones permiten reducir procesamiento innecesario sin sacrificar trazabilidad, resiliencia ni control sobre acciones críticas.